Software Requirements Specification (SRS)

PROJECT TITLE: Momentum

PROJECT DEVELOPER: Gavino Vargas

PROJECT REPOSITORY (GITHUB LINK): https://github.com/gavinov10/momentum.git

Table of Contents

1. **Revision History**

2. **Introduction**
   - 2.1 Purpose
   - 2.2 Document Conventions
   - 2.3 Intended Audience and Reading Suggestions
   - 2.4 Project Scope
   - 2.5 References

3. **Overall Description**
   - 3.1 Product Perspective
   - 3.2 Product Features
   - 3.3 User Classes and Characteristics
   - 3.4 Operating Environment
   - 3.5 Design and Implementation Constraints
   - 3.6 User Documentation
   - 3.7 Assumptions and Dependencies

4. **System Features**
   - 4.1 User Account Management
   - 4.2 Job Application Creation
   - 4.3 Automatic Email Parsing
   - 4.4 Application Status Tracking
   - 4.5 Timeline and Activity View
   - 4.6 Notes and Comments
   - 4.7 Google Sheets / Notion Sync
   - 4.8 Search and Filtering
   - 4.9 Dashboard Overview
   - 4.10 Email-Based Activity Logging

5. **External Interface Requirements**
   - 5.1 User Interfaces
   - 5.2 Hardware Interfaces

5.3 Software Interfaces
5.4 Communications Interfaces

6. Nonfunctional Requirements
6.1 Performance Requirements
6.2 Safety Requirements
6.3 Security Requirements
6.4 Software Quality Attributes

7. Other Requirements

Appendix A – Glossary
Appendix B – Analysis Models
Appendix C – Issues List

Revision History

Name: Gavino Vargas	Date: 14 Nov. 25	Reason for Changes	Version

1. Introduction

This section introduces the purpose, scope, and audience of this SRS. It must be consistent with the Project Charter submitted earlier.

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and nonfunctional requirements for Momentum, version 1.0. Momentum is a web-based job application tracking system designed to automatically organize, monitor, and update a user's job or internship applications. The system helps users streamline their application process by centralizing key information such as company details, role applied for, status updates, and communication history. This SRS defines the features, constraints, and interfaces needed for successful development.

1.2 Document Conventions

This document follows standard SRS structure based on IEEE guidelines. Section headings are numbered for clarity. Keywords such as shall, must, and required indicate mandatory requirements, while may and optional indicate non-mandatory items. Code elements, API names, and database fields appear in monospace formatting.

1.3 Intended Audience and Reading Suggestions

This document is intended for developers, testers, project evaluators, and instructors reviewing the Momentum system.

- Developers: should read the full document, focusing on Sections 3–6.
- Testers: should prioritize Sections 3 and 5.
- Instructors & reviewers: may focus on Sections 1 - 4 for conceptual understanding.
  All readers may review the appendices for reference diagrams and definitions.

1.4 Project Scope

Momentum provides users with a centralized dashboard to track job applications and automatically extract updates from email notifications. The platform improves clarity, reduces manual record-keeping, and supports better decision-making during application

cycles. The system benefits students, job seekers, and early-career professionals seeking a more efficient application workflow..

1.5 References

The following are references to Momentum:

- Momentum Project Charter
- FastAPI Documentation (https://fastapi.tiangolo.com)
- SQLite Documentation
- Google API Documentation (for Gmail and Sheets integration)
- IEEE Software Requirements Specification Guidelines

2. Overall Description

2.1 Product Perspective

Momentum is a web-application designed to help users organize and track their job and internship applications. It integrates with external services, such as Gmail and optionally Google Sheets to automatically detect new application activity and update the user's records. Momentum follows a client–server architecture with a Python/FastAPI backend and a modern web frontend.

2.2 Product Features

In a high-level, Momentum provides the following core capabilities:

- Create and store job application entries
- Automatically extract updates from email notifications
- Track application statuses (applied, interview, offer, rejection, etc.)
- Allow users to add notes or comments
- Provide a searchable, filterable view of all applications
- Sync data to an external sheet or workspace (optional feature)
- Display a dashboard summarizing activity and timelines

2.3 User Classes and Characteristics

Momentum is intended for:

- Primary Users: Students and job seekers who submit multiple applications and need an organized tracking tool
- Secondary Users: Instructors or career advisors reviewing student progress (optional)
  Users are assumed to have basic computer literacy and access to email.

2.4 Operating Environment

Frontend: Modern web browsers (Chrome, Safari, Firefox, Edge)

Backend: FastAPI with SQLAlchemy ORM running on macOS, Linux, or cloud environments

Database: SQLite during development; PostgreSQL when deployed

APIs: Gmail API and optional Google Sheets

Devices: Laptop or desktop computer with internet connection

2.5 Design and Implementation Constraints

- SQLAlchemy ORM must be used for all database access
- Gmail API requires OAuth2 authentication and user permission
- SQLite's concurrency limitations require PostgreSQL for production use
- External integrations (Sheets) may impose rate limits or usage quotas
- The system must comply with privacy and security expectations for handling email content
- Only modern browsers are supported; legacy browsers may not function correctly

2.6 User Documentation

This will include:

- A user onboarding and setup guide
- Instructions for connecting Gmail via OAuth
- Steps for creating, viewing, and updating applications
- FAQ and troubleshooting information

2.7 Assumptions and Dependencies

- Users have an active Gmail account and grant permission for email scanning
- Email notifications from companies follow identifiable patterns
- Stable internet connection is required for major features
- External APIs (Sheets) function as expected
- Future enhancements may expand automation and integration behaviors

3. System Features

3.1 User Account Management

Description and Priority

Allows users to create an account, log in, and manage their profile.
Priority: High — required for personalized tracking.

Stimulus/Response Sequences

- Stimulus: User submits signup form.
  Response: System validates input and creates a new user profile.
- Stimulus: User logs in.
  Response: System authenticates credentials and grants access.

Functional Requirements

- The system shall allow users to create an account with name and email.
- The system shall authenticate users before granting access.
- The system shall store user information securely.

3.2 Job Application Creation

Description and Priority

Allows users to add a new job application record manually.
Priority**:** High.

Stimulus/Response Sequences

- Stimulus: User submits a new job application form.
  Response*:* System validates and stores the application in the database.

Functional Requirements

- The system shall allow users to create application entries.
- The system shall require company name and role fields.
- The system shall store optional fields (company size, date applied, notes).

3.3 Automatic Email Parsing

Description and Priority

Automatically scans authorized Gmail accounts for job-related emails and extracts updates.
Priority: High — key differentiating feature.

Stimulus/Response Sequences

- Stimulus: The system detects a new email.
  Response: The system analyzes it and extracts meaningful application updates.

Functional Requirements

- The system shall connect to Gmail using OAuth2.
- The system shall identify emails related to applications (confirmations, interview invites, rejections).
- The system shall update the corresponding application record.

3.4 Application Status Tracking

Description and Priority

Tracks the progress of job applications (Applied, OA, Interview, Offer, Rejected).
Priority: High.

Stimulus/Response Sequences

- Stimulus: Application receives new information via email or user input.
  Response: Status is updated automatically or manually.

Functional Requirements

- The system shall support predefined status values.
- The system shall update status when triggered by an email event.
- The system shall permit manual status changes.

3.5 Timeline and Activity View

Description and Priority

Displays activities for each application in a chronological timeline.
Priority: Medium.

Stimulus/Response Sequences

- Stimulus: User opens application details.
  Response: System displays timeline of updates.

Functional Requirements

- The system shall record timestamps for updates.
- The system shall display activities in chronological order.

3.6 Notes and Comments

Description and Priority

Allows users to attach notes to each application.
Priority: Medium.

Stimulus/Response Sequences

- Stimulus: User enters a note on an application page.
  Response: System saves and displays the note.

Functional Requirements

- The system shall allow users to create, edit, and delete notes.
- The system shall store notes linked to the appropriate application.

3.7 External Sync (Google Sheets)

Description and Priority

Syncs application data with Google Sheets or Notion for optional external viewing.
Priority: Low–Medium (optional enhancement).

Stimulus/Response Sequences

- Stimulus: User enables syncing.
  Response: System sends updated application records to external sheet/workspace.

Functional Requirements

- The system shall allow users to toggle external sync.
- The system shall update external records when application data changes.

3.8 Search and Filtering

Description and Priority

Allows users to search and filter applications by status, company, role, or date.
Priority: Medium.

Stimulus/Response Sequences

- Stimulus: User enters a search term or applies filters.
  Response: System returns matching applications.

Functional Requirements

- The system shall allow searching by company name, role, and notes.
- The system shall filter by status, date applied, and activity date.

3.9 Dashboard Overview

Description and Priority

Displays a high-level summary of the user's application activity.
Priority: Medium.

Stimulus/Response Sequences

- Stimulus: User opens dashboard.
  Response: System displays counts, charts, and recent updates.

Functional Requirements

- The system shall show total applications and counts per status.
- The system shall display recent activity in the dashboard.

3.10 Email-Based Activity Logging

Description and Priority

Logs actions such as received emails, detected changes, and updates for internal transparency.
Priority: Low–Medium.

Stimulus/Response Sequences

- Stimulus: Email parser detects relevant message.
  Response: System logs the event internally.

Functional Requirements

- The system shall maintain an internal log of email events.
- The system shall allow developers/testers to access this log for debugging.

4. External Interface Requirements

4.1 User Interfaces

Momentum will be delivered as a web-based application accessible through modern browsers. Key user interface components include:

- **Login and Authorization Screen:** Allows users to sign in and connect their Gmail account via OAuth2.
- **Dashboard View:** Displays a summary of job applications, activity counts, recent updates, and quick filters.
- **Application List Page:** Shows all stored applications with options to search, filter, and sort.
- **Application Detail Page:** Displays status, company information, notes, and timeline of updates.
- **Application Creation Page:** Form for adding new applications manually.
- **Settings Page:** Allows users to manage email sync, external sync (Google Sheets), and profile settings.
- **Responsive Layout:** Designed to function on laptops and desktops; mobile support is optional for future releases.

The UI will use a clean, accessible layout with consistent typography and intuitive interaction patterns.

4.2 Hardware Interfaces

Momentum is a cloud-based application and does not directly require specialized hardware. The following hardware environment is assumed to have:

- **Client Devices:** Laptop or desktop computer capable of running a modern web browser.
- **Server Hardware:** Any cloud or local machine capable of running FastAPI, SQLAlchemy, and the database engine (SQLite or PostgreSQL).

No direct hardware integration (sensors, IoT, peripherals) is required.

4.3 Software Interfaces

Momentum interacts with several software systems and services:

Backend Interfaces

- **FastAPI Framework:** Handles all routing, authentication, and API endpoints.
- **SQLAlchemy ORM:** Manages database interactions and relational mappings.
- **Database Engine:**
  - SQLite for development
  - PostgreSQL for deployment

External API Interfaces

- **Gmail API:**
  - OAuth2 authentication
  - Reading messages for parsing
  - Requires client credentials and user authorization
- **Google Sheets API (Optional):**
  - Writes updated application entries to an external spreadsheet
  - Requires API keys and OAuth scopes

Frontend Interfaces

- Communicates with backend via RESTful JSON APIs (HTTPS).
- Uses Axios or Fetch API for request handling.

4.4 Communications Interfaces

Momentum uses standard internet-based communication protocols:

- **HTTPS:** All client-to-server communication must be secured with HTTPS to protect user data.

- **RESTful API Calls:** Frontend sends JSON requests to backend endpoints for creating, reading, updating, and deleting data.
- **OAuth2 Protocol:** Used for secure authentication with Gmail and optional external services.
- **JSON Messaging:** Primary format for all data sent between frontend and backend.
- **Email Retrieval:** Gmail API uses HTTP requests, OAuth2 tokens, and Google's message retrieval endpoints.

Network requirements:

- Stable internet connection for email syncing and API interactions.
- Server must support CORS configuration to allow frontend communication.

5. Other nonfunctional Requirements

5.1 Performance Requirements

- The system shall respond to user actions (e.g., viewing applications, updating status) within **2 seconds** under normal load.
- The system shall be able to process and parse emails in the background without interrupting user interactions.
- The backend shall handle up to **100 application records per user** without noticeable performance degradation.
- Database queries shall complete within **500 ms** for typical operations (search, filtering, listing).
- Sync operations (Sheets) shall complete within a reasonable time, depending on external API performance.

5.2 Safety Requirements

- The system shall not modify or delete any emails in the user's Gmail account.
- The system shall handle authentication failures or expired tokens gracefully without data corruption.

- The system shall prevent unintended overwriting of application data during sync operations by validating fields before applying updates.
- The system shall ensure that database operations use transactions to avoid partial updates in case of failures.

5.3 Security Requirements

- All data exchanged between client and server shall be encrypted using **HTTPS**.
- OAuth2 authentication must be used for Gmail and external integrations to ensure secure access.
- User passwords (if applicable) shall be stored using industry-standard hashing algorithms (e.g., bcrypt).
- The system shall restrict users from accessing or modifying data belonging to other users.
- API endpoints shall validate all inputs to prevent injection attacks.
- Access tokens and API keys shall be stored securely using environment variables.
- Sessions shall expire after a predetermined period of inactivity.

5.4 Software Quality Attributes

Reliability

- The system shall maintain consistent uptime during typical usage.
- Email parsing tasks should be resilient to network delays or rate limits.

Usability

- The interface shall be simple, clean, and easy to navigate for users of varying technical skill levels.
- Important actions like adding applications or checking updates shall require minimal clicks.

Maintainability

- The backend shall use a modular architecture (separate layers for routes, services, models).
- Code shall follow consistent naming conventions and formatting guidelines.
- SQLAlchemy shall be used for all database access to simplify updates and migrations.

Scalability

- The system architecture shall support future migration from SQLite to PostgreSQL for increased concurrency.
- Background email processing can scale with Celery or similar tools in future releases.

Portability

- The web application shall run on any modern operating system capable of hosting FastAPI and a Python runtime.
- The frontend shall function on all major browsers without special installation.

6. Other Requirements

6.1 Legal and Regulatory Requirements

- The system shall comply with Google's API Terms of Service, particularly regarding permissible Gmail data access and storage.
- User data shall be handled in compliance with general privacy expectations, ensuring no unauthorized sharing or misuse of personal information.
- Momentum shall only access email metadata and job-related content for the purpose of updating application records; no other email content shall be stored or processed beyond what is required.

6.2 Database Requirements

- SQLite shall be used during development for simplicity; PostgreSQL is recommended for production due to improved concurrency, security, and scaling capability.
- Database schemas shall be managed using SQLAlchemy and Alembic migrations to ensure maintainability.

- The database must enforce user-to-application relationships using foreign key constraints to preserve data integrity.

6.3 Internationalization Requirements

- The system shall use English as the default language.
- Dates and timestamps shall be stored in standardized UTC format to support future internationalization.
- Future versions may support localization of time zones and date formats.

6.4 Reusability and Extensibility Requirements

- The system's modular architecture (routes, services, schemas, ORM models) shall allow components to be reused or extended.
- The email parsing logic shall be designed so additional email providers (e.g., Outlook) can be added in future versions.
- The data syncing system shall support switching between various integrations (Google Sheets, Notion, Airtable) with minimal code changes.
- The application status tracking model shall allow new status types to be added without breaking existing functionality.

6.5 Deployment Considerations

- Environment variables must be used to store sensitive information such as API keys, OAuth credentials, and database URLs.
- Deployment environment should support HTTPS and have firewall or security rules configured to protect API endpoints.
- The system should be deployable on cloud-hosting platforms such as Render, Railway, or AWS.

Appendix A: Glossary

API (Application Programming Interface)
A set of rules that allows software components to communicate. Momentum uses APIs such as Gmail API and Google Sheets API.

Application Entry
A record created by the user or automatically detected by the system that contains details about a job or internship application.

Application Status
A predefined label representing the user's progress in the application process (e.g., Applied, Interview, Offer, Rejected).

Backend
The server-side logic of the application, including FastAPI, SQLAlchemy, and the database.

Dashboard
A user interface that displays an overview of application statistics, recent updates, and summaries.

Database
A structured storage system for application and user data. Momentum uses SQLite for development and PostgreSQL for deployment.

Email Parsing
The process of automatically reading and interpreting email content to update application statuses.

FastAPI
A Python web framework used to build Momentum's backend API.

Gmail API
A Google service that allows authorized applications to securely read email messages from a user's Gmail account.

JSON (JavaScript Object Notation)
A lightweight data format used to exchange information between the front end and back end.

Momentum
The name of the software system described in this SRS—an application tracking platform for job seekers.

OAuth2
A secure protocol used for authorization. Momentum uses OAuth2 for Gmail access and external integrations.

ORM (Object-Relational Mapping)
A technique that allows developers to interact with the database using Python objects. SQLAlchemy is the ORM used in this project.

SQLAlchemy
A Python ORM library used to define database models and run queries for Momentum.

Syncing (Synchronization)
The process of copying or updating data between Momentum and an external service (e.g., Google Sheets or Notion).

Timeline
A chronological display of activity or updates related to a specific job application.

User Interface (UI)
Everything the user interacts with visually on the frontend of the application.

Appendix B: Analysis Models

B.1 Use Case Diagram (Textual Representation)

Actors:

- **User** (Primary actor)
- **Gmail API** (External system)
- **Google Sheets / Notion API** (Optional external system)

Use Cases:

1. **Create Account** – User registers and creates a new profile
2. **Log In** – User authenticates
3. **Add Application** – User manually creates an entry
4. **View Applications** – User views list of applications
5. **Update Application** – Modify role, status, notes, etc.
6. **Connect Gmail** – User authorizes Gmail access
7. **Automatic Email Parsing** – System scans Gmail for relevant updates

8. **Update Application Status Automatically** – Email changes trigger status updates

9. **Sync with Google Sheets / Notion** – Optional export and synchronization

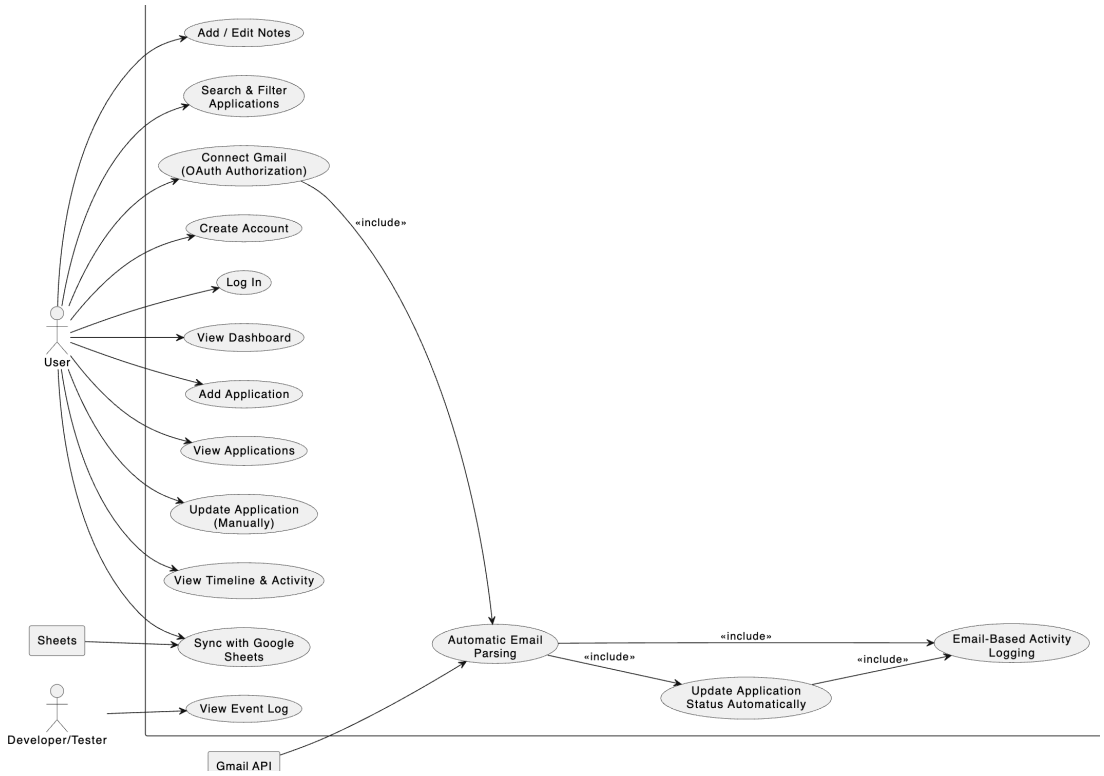
10. **View Dashboard** – User views summary of application activity

B.2 Entity–Relationship (ER) Diagram (Textual Representation)

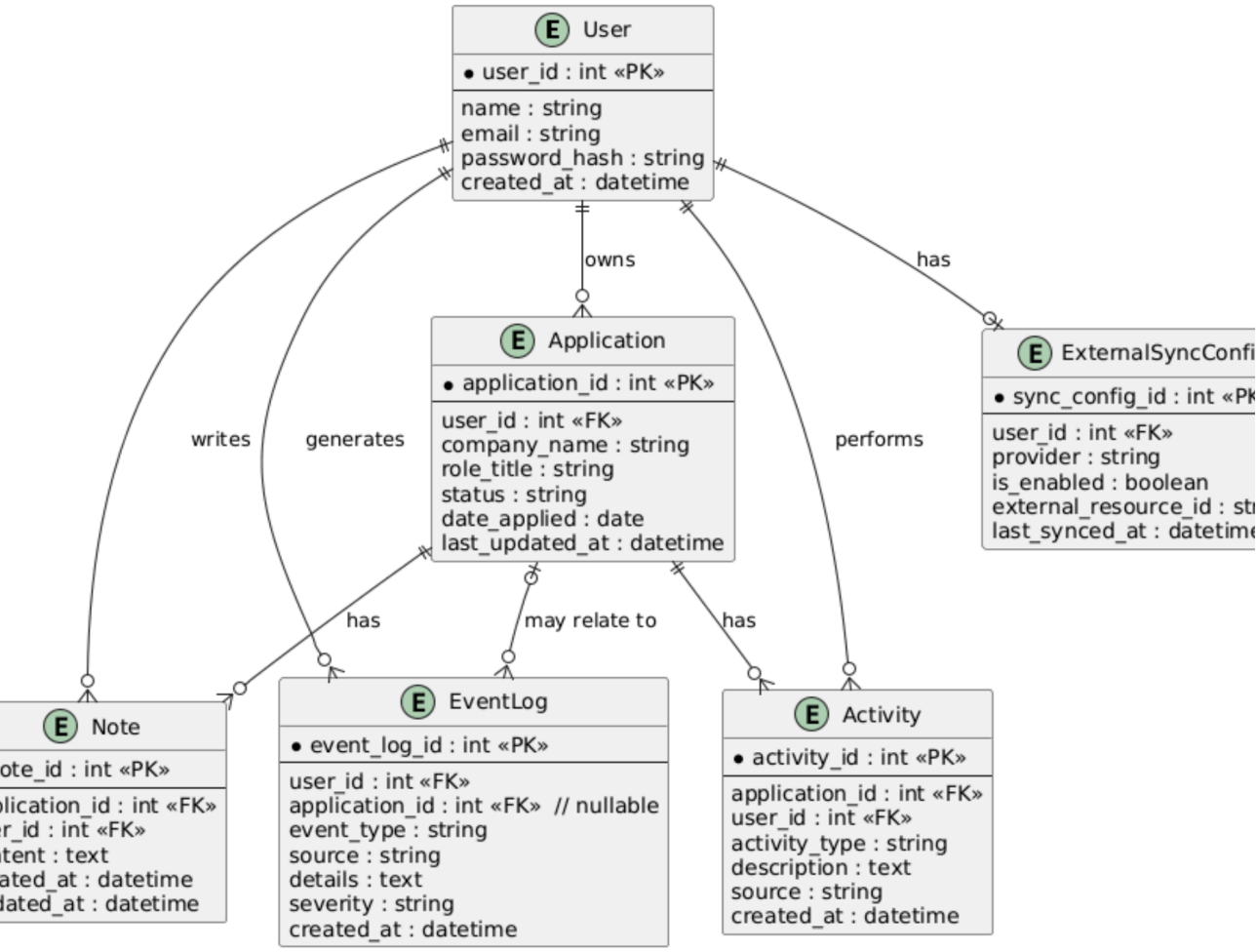
This ER model reflects your SQLAlchemy models (User, Application) and their relationships.

B.3 Object-Oriented Class Diagram (Text-Based)

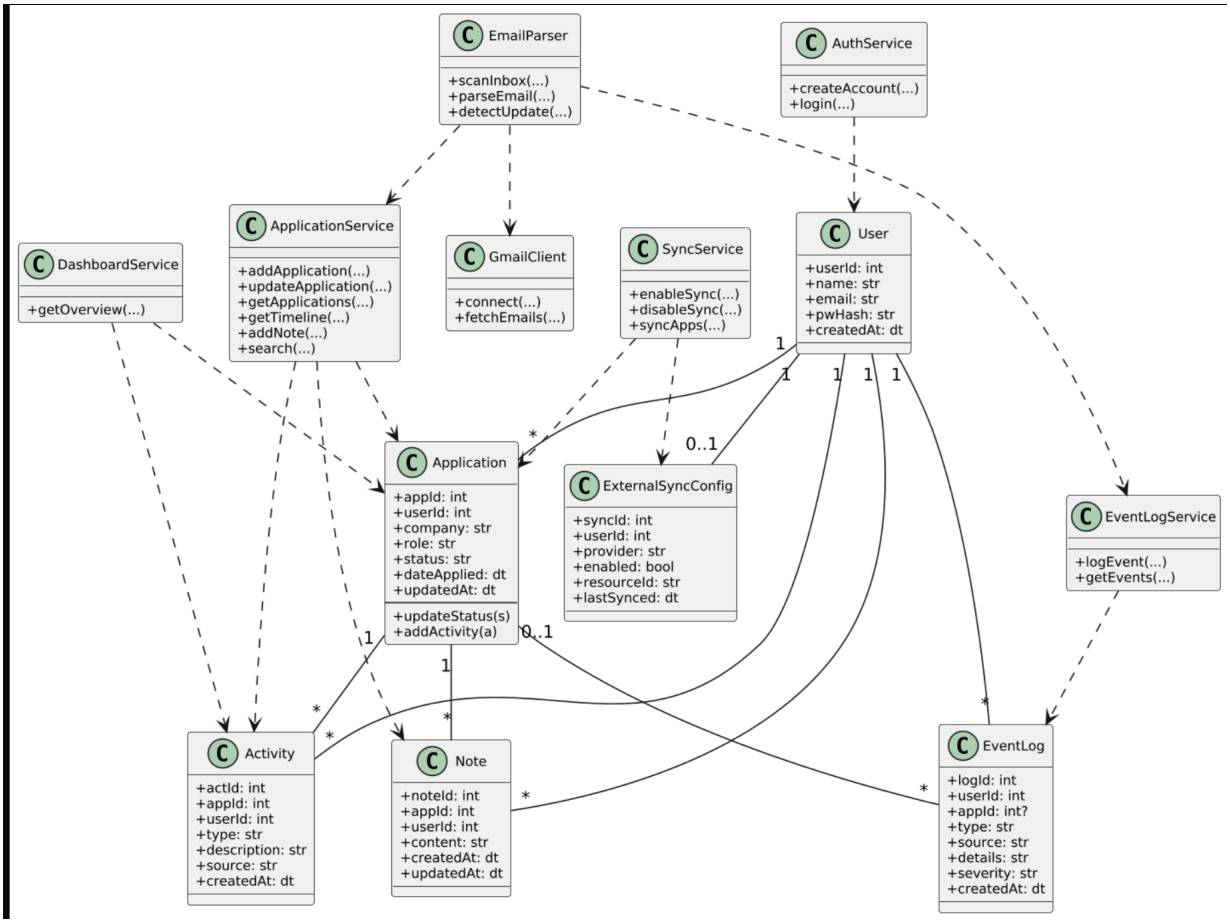
Reflects how your Python classes (SQLAlchemy models) interact.



Use Case Diagram



Entity Relationship



Objected Oriented Class Diagram